

Capítulo 10: Sistemas Populacionais

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

22 de junho de 2026

Sumário

- 1 Introdução
- 2 Modelos de Crescimento Populacional
- 3 Interações entre Espécies
- 4 Modelos Epidemiológicos
- 5 Dinâmica Espacial
- 6 Exercícios
- 7 Referências

Visão Geral do Capítulo

Objetivos de Aprendizagem

- Compreender modelos de dinâmica populacional
- Analisar interações entre espécies
- Estudar modelos epidemiológicos
- Explorar dinâmica espacial de populações
- Implementar simulações em Python

Por que Estudar Sistemas Populacionais?

- Conservação de espécies ameaçadas
- Controle de epidemias
- Gestão de recursos naturais
- Compreensão de ecossistemas

Definição: Sistema Populacional

Conjunto de indivíduos da mesma espécie (ou espécies interagentes) cuja densidade varia no tempo e espaço devido a processos de nascimento, morte, migração e interações.

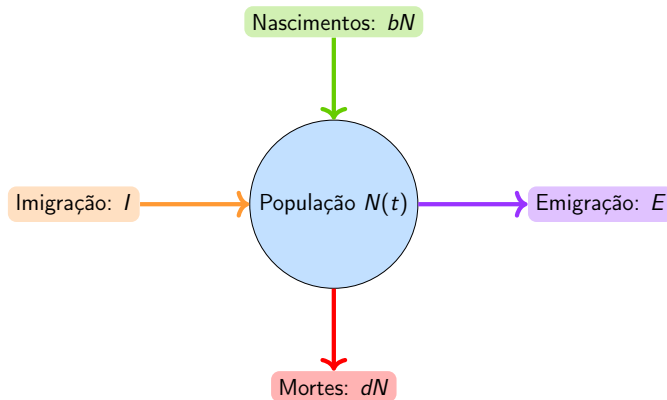
Níveis de Organização:

- **Espécie única:** crescimento, regulação
- **Interações:** predação, competição, mutualismo
- **Espacial:** metapopulações, difusão

Aplicações:

- Ecologia teórica
- Epidemiologia
- Biologia evolutiva
- Gestão ambiental

Processos Fundamentais



$$\frac{dN}{dt} = \underbrace{bN}_{\text{nascimento}} - \underbrace{dN}_{\text{morte}} + \underbrace{I}_{\text{imigração}} - \underbrace{E}_{\text{emigração}}$$

Crescimento Exponencial (Malthusiano)

Modelo Mais Simples

População cresce proporcionalmente ao seu tamanho:

$$\frac{dN}{dt} = rN$$

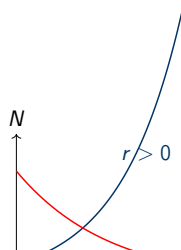
onde $r = b - d$ é a taxa intrínseca de crescimento per capita.

Solução analítica:

$$N(t) = N_0 e^{rt}$$

Características:

- Crescimento ilimitado ($r > 0$)
- Decaimento exponencial ($r < 0$)
- Tempo de duplicação: $t_d = \frac{\ln 2}{r}$



Crescimento Logístico (Verhulst)

Modelo com Capacidade de Suporte

Incorpora limitação de recursos através da capacidade de suporte K :

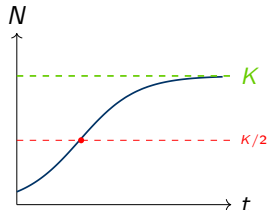
$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right)$$

Solução analítica:

$$N(t) = \frac{KN_0 e^{rt}}{K + N_0(e^{rt} - 1)} = \frac{K}{1 + \left(\frac{K - N_0}{N_0} \right) e^{-rt}}$$

Propriedades:

- Forma sigmoïdal (curva S)
- $N \rightarrow K$ quando $t \rightarrow \infty$
- Ponto de inflexão em $N = K/2$
- Taxa máxima em $N = K/2$



Implementação: Crescimento Logístico

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def logistic(N, t, r, K):
6     """Modelo de crescimento logístico"""
7     return r * N * (1 - N/K)
8
9 # Parametros
10 r = 0.8      # taxa de crescimento
11 K = 1000     # capacidade de suporte
12 N0 = 50      # populacao inicial
13
14 # Tempo
15 t = np.linspace(0, 20, 500)
16
17 # Resolver EDO
18 N = odeint(logistic, N0, t, args=(r, K))
19
20 # Plotar
21 plt.figure(figsize=(10, 6))
22 plt.plot(t, N, 'b-', linewidth=2, label='N(t)')
23 plt.axhline(y=K, color='r', linestyle='--',
24             linewidth=2, label=f'K = {K}')
25 plt.xlabel('Tempo')
26 plt.ylabel('Populacao')
27 plt.title('Crescimento Logístico')
28 plt.legend()
29 plt.grid(True, alpha=0.3)
```

Listing 1: Modelo logístico em Python

Efeito Allee

Crescimento com Densidade Crítica

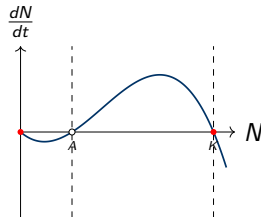
Em algumas populações, baixas densidades reduzem a taxa de crescimento (dificuldade de encontrar parceiros, defesa contra predadores):

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right) \left(\frac{N}{A} - 1\right)$$

onde A é a densidade crítica (limiar de Allee).

Características:

- Três equilíbrios: $N^* = 0, A, K$
- $N = 0$ estável, $N = A$ instável
- $N = K$ estável
- Risco de extinção se $N < A$



Modelos em Tempo Discreto

Definição: Equações de Diferença

Modelos onde o tempo avança em passos discretos: $N_{t+1} = f(N_t)$

Modelo de Ricker

$$N_{t+1} = N_t e^{r(1-N_t/K)}$$

Características:

- Pode exibir caos
- Usado para peixes com gerações discretas
- Comportamento complexo para r alto

Modelo de Beverton-Holt

$$N_{t+1} = \frac{RN_t}{1 + (R - 1)N_t/K}$$

Características:

- Sempre estável
- R = taxa de reprodução líquida
- Converge monotonicamente para K

Implementação: Modelo de Ricker

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def ricker(N, r, K):
5     """Modelo de Ricker"""
6     return N * np.exp(r * (1 - N/K))
7
8 # Parametros
9 r_values = [1.5, 2.0, 2.7] # diferentes taxas de crescimento
10 K = 100
11 N0 = 10
12 t_max = 50
13
14 # Simular
15 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
16
17 for i, r in enumerate(r_values):
18     N = np.zeros(t_max)
19     N[0] = N0
20
21     for t in range(t_max - 1):
22         N[t+1] = ricker(N[t], r, K)
23
24     axes[i].plot(N, 'b.-', markersize=4)
25     axes[i].axhline(y=K, color='r', linestyle='--',
26                     label=f'K = {K}')
27     axes[i].set_xlabel('Geracao t')
28     axes[i].set_ylabel('N(t)')
29     axes[i].set_title(f'r = {r}')
30     axes[i].legend()
31     axes[i].grid(True, alpha=0.3)
32
33 plt.tight_layout()
```

Listing 2: Dinâmica discreta com Ricker

Diagrama de Linha de Fase

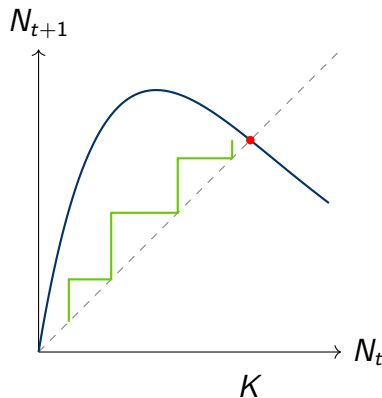
Análise Gráfica:

- Plotar N_{t+1} vs N_t
- Linha de identidade: $N_{t+1} = N_t$
- Equilíbrios: intersecções
- Estabilidade: inclinação

Critério de Estabilidade:

$$\left| \frac{df}{dN} \right|_{N^*} < 1$$

Se > 1 : instável (oscilações ou caos)



Análise Cobweb: trajetória iterativa

Tipos de Interações

Interação	Espécie 1	Espécie 2	Exemplo
Competição	—	—	Leões vs Hienas
Predação	+	—	Lobo vs Coelho
Mutualismo	+	+	Abelha vs Flor
Comensalismo	+	0	Rêmora vs Tubarão
Amensalismo	—	0	Árvore vs Plântula
Parasitismo	+	—	Carrapato vs Cão

Notação

- (+) = efeito benéfico na taxa de crescimento
- (—) = efeito prejudicial na taxa de crescimento
- (0) = sem efeito

Modelo Lotka-Volterra Predador-Presa

Sistema de Equações

$$\frac{dV}{dt} = \alpha V - \beta VP \quad (\text{Presas})$$

$$\frac{dP}{dt} = \delta \beta VP - \gamma P \quad (\text{Predadores})$$

Parâmetros:

- V = densidade de presas
- P = densidade de predadores
- α = taxa de crescimento das presas
- β = taxa de predação
- δ = eficiência de conversão (biomassa presa $\rightarrow$ predador)
- γ = taxa de morte dos predadores

Interpretação Biológica

Equação das Presas:

$$\frac{dV}{dt} = \underbrace{\alpha V}_{\text{crescimento exponencial}} - \underbrace{\beta VP}_{\text{perda por predação}}$$

- Sem predadores: $\frac{dV}{dt} = \alpha V$
- Taxa de encontro: lei de ação das massas (VP)
- Predação reduz crescimento

Equação dos Predadores:

$$\frac{dP}{dt} = \underbrace{\delta \beta VP}_{\text{crescimento por consumo}} - \underbrace{\gamma P}_{\text{morte natural}}$$

- Sem presas: $\frac{dP}{dt} = -\gamma P$
- Crescimento proporcional ao consumo
- Morte independente de densidade

Importante!

Este sistema exibe oscilações periódicas permanentes!

Análise de Equilíbrio

Pontos de Equilíbrio

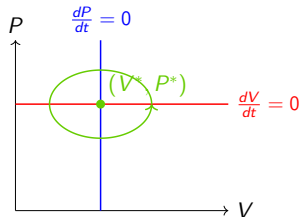
Resolver $\frac{dV}{dt} = 0$ e $\frac{dP}{dt} = 0$:

1. **Extinção total:** $(V^*, P^*) = (0, 0)$
2. **Coexistência:**

$$V^* = \frac{\gamma}{\delta\beta}, \quad P^* = \frac{\alpha}{\beta}$$

Isoclinas nulas:

- **Presas:** $\frac{dV}{dt} = 0 \Rightarrow V = 0$ ou $P = \alpha/\beta$
- **Predadores:** $\frac{dP}{dt} = 0 \Rightarrow P = 0$ ou $V = \gamma/(\delta\beta)$



Implementação: Lotka-Volterra

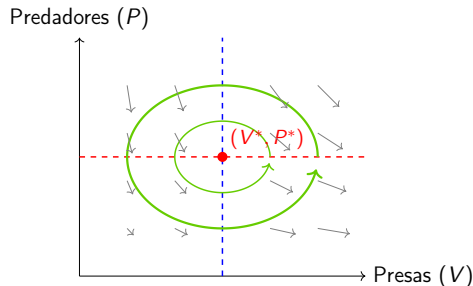
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def lotka_volterra(y, t, alpha, beta, delta, gamma):
6     """Sistema Lotka-Volterra predador-presa"""
7     V, P = y
8     dVdt = alpha*V - beta*V*P
9     dPdt = delta*beta*V*P - gamma*P
10    return [dVdt, dPdt]
11
12 # Parametros
13 alpha = 1.0    # crescimento presas
14 beta = 0.1     # taxa de predacao
15 delta = 0.75  # eficiencia conversao
16 gamma = 1.5   # morte predadores
17
18 # Condições iniciais
19 y0 = [40, 9]   # [V0, P0]
20 t = np.linspace(0, 50, 1000)
21
22 # Resolver
23 sol = odeint(lotka_volterra, y0, t,
24             args=(alpha, beta, delta, gamma))
25
26 # Plotar series temporais
27 plt.figure(figsize=(12, 5))
28 plt.subplot(1, 2, 1)
29 plt.plot(t, sol[:, 0], 'b-', linewidth=2, label='Presas (V)')
30 plt.plot(t, sol[:, 1], 'r-', linewidth=2, label='Predadores (P)')
31 plt.xlabel('Tempo')
32 plt.ylabel('Densidade')
33 plt.legend()
34 plt.grid(True, alpha=0.3)
```

Listing 3: Sistema predador-presa

Retrato de Fase

```
1 # Retrato de fase
2 plt.subplot(1, 2, 2)
3 plt.plot(sol[:, 0], sol[:, 1],
4         'g-', linewidth=2)
5 plt.plot(sol[0, 0], sol[0, 1],
6         'go', markersize=10,
7         label='Início')
8
9 # Equilíbrio
10 V_eq = gamma/(delta*beta)
11 P_eq = alpha/beta
12 plt.plot(V_eq, P_eq, 'r*',
13         markersize=15,
14         label='Equilíbrio')
15
16 # Isoclinas
17 V_range = np.linspace(0, 60, 100)
18 plt.axhline(y=P_eq, color='r',
19             linestyle='--',
20             alpha=0.5,
21             label='dV/dt=0')
22 plt.axvline(x=V_eq, color='b',
23             linestyle='--',
24             alpha=0.5,
25             label='dP/dt=0')
26
27 plt.xlabel('Presas (V)')
28 plt.ylabel('Predadores (P)')
29 plt.legend(fontsize=8)
30 plt.grid(True, alpha=0.3)
31 plt.tight_layout()
```

Listing 4: Plano de fase



Observações:

- Órbitas fechadas (ciclos)
- Período depende de CI
- Centro neutro (não atrator)

Competição: Lotka-Volterra

Sistema de Competição

Duas espécies competindo pelos mesmos recursos:

$$\begin{aligned}\frac{dN_1}{dt} &= r_1 N_1 \left(1 - \frac{N_1 + \alpha_{12} N_2}{K_1} \right) \\ \frac{dN_2}{dt} &= r_2 N_2 \left(1 - \frac{N_2 + \alpha_{21} N_1}{K_2} \right)\end{aligned}$$

Coefficientes de competição:

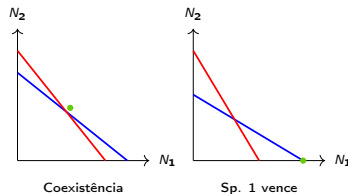
- α_{12} = efeito da espécie 2 sobre a espécie 1
- α_{21} = efeito da espécie 1 sobre a espécie 2

Interpretação: $\alpha_{12} N_2$ converte indivíduos da espécie 2 em "equivalentes" da espécie 1 em termos de uso de recursos.

Exclusão Competitiva

Quatro Cenários Possíveis

1. **Espécie 1 vence:** $K_1 > \alpha_{12}K_2$ e $K_2 < \alpha_{21}K_1$
2. **Espécie 2 vence:** $K_2 > \alpha_{21}K_1$ e $K_1 < \alpha_{12}K_2$
3. **Coexistência estável:** $K_1 > \alpha_{12}K_2$ e $K_2 > \alpha_{21}K_1$
4. **Exclusão dependente de CI:** $K_1 < \alpha_{12}K_2$ e $K_2 < \alpha_{21}K_1$



Importante!

Princípio da Exclusão Competitiva (Gause): duas espécies não podem coexistir no mesmo nicho ecológico.

Mutualismo

Modelo de Mutualismo

Interação benéfica mútua (polinizadores-plantas, micorrizas-raízes):

$$\begin{aligned}\frac{dN_1}{dt} &= r_1 N_1 \left(1 - \frac{N_1}{K_1 + \alpha_{12} N_2} \right) \\ \frac{dN_2}{dt} &= r_2 N_2 \left(1 - \frac{N_2}{K_2 + \alpha_{21} N_1} \right)\end{aligned}$$

Características:

- Coeficientes $\alpha_{ij} > 0$ aumentam capacidade de suporte
- Mutualismo facultativo: espécies podem sobreviver sozinhas
- Mutualismo obrigatório: requer ambas espécies

Exemplo: Abelhas e Flores

Abelhas aumentam reprodução das plantas (polinização)

Flores fornecem néctar/pólen para abelhas

Coevolução: dependência mútua crescente

Modelo Nicholson-Bailey

Hospedeiro-Parasitóide em Tempo Discreto

$$H_{t+1} = \lambda H_t e^{-aP_t}$$

$$P_{t+1} = cH_t(1 - e^{-aP_t})$$

Parâmetros:

- H_t = densidade de hospedeiros na geração t
- P_t = densidade de parasitóides
- λ = taxa de reprodução do hospedeiro
- a = taxa de ataque do parasitóide
- c = número de parasitóides emergindo por hospedeiro

Distribuição de Poisson: e^{-aP_t} é a probabilidade de um hospedeiro escapar do parasitismo.

Aplicação

Controle biológico de pragas agrícolas

Definição: Modelo Epidemiológico

Modelo matemático que descreve a transmissão e propagação de doenças infecciosas em uma população.

Compartimentos:

- **S:** Suscetíveis
- **E:** Expostos (incubação)
- **I:** Infectados (infecciosos)
- **R:** Recuperados (imunes)
- **D:** Mortos

Aplicações:

- Previsão de epidemias
- Avaliação de intervenções
- Políticas de vacinação
- Alocação de recursos

Exemplos Históricos

COVID-19, Influenza, Ebola, HIV/AIDS, Malária, Dengue

Modelo SIR

Sistema SIR Clássico

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} \\ \frac{dI}{dt} &= \beta \frac{SI}{N} - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

Parâmetros:

- β = taxa de transmissão (contatos infectantes por unidade de tempo)
- γ = taxa de recuperação ($1/\gamma$ = período infeccioso)
- $N = S + I + R$ = população total (constante)



Número Básico de Reprodução

Definição: R_0 (R-naught)

Número médio de infecções secundárias causadas por um indivíduo infectado introduzido em uma população completamente susceptível.

$$R_0 = \frac{\beta}{\gamma} = \beta \times \frac{1}{\gamma} = (\text{taxa de contato}) \times (\text{duração da infecção})$$

Interpretação Epidemiológica

- $R_0 < 1$: epidemia se extingue (cada infectado infecta < 1 pessoa)
- $R_0 = 1$: limiar epidêmico (endemia estável)
- $R_0 > 1$: epidemia se propaga

Valores Típicos de R_0

- Sarampo: 12-18 (altamente contagioso)
- COVID-19 (original): 2-3
- Influenza sazonal: 1-2
- Ebola: 1.5-2.5

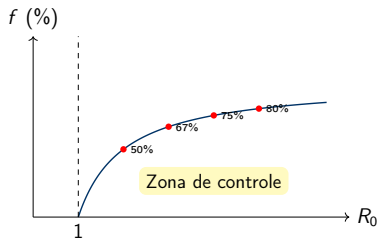
Limiar de Vacinação

Fração Crítica de Vacinação

Para eliminar a doença, é necessário vacinar uma fração f da população:

$$f > 1 - \frac{1}{R_0}$$

Derivação: Vacinação reduz suscetíveis efetivos. Para controle, precisamos $R_0^{eff} = R_0(1 - f) < 1$.



Exemplo: Sarampo ($R_0 \approx 15$)

$$f > 1 - 1/15 = 0.93 \text{ (93\% de cobertura vacinal necessária)}$$

Implementação: Modelo SIR

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def sir_model(y, t, beta, gamma, N):
6     """Modelo SIR"""
7     S, I, R = y
8     dSdt = -beta * S * I / N
9     dIdt = beta * S * I / N - gamma * I
10    dRdt = gamma * I
11    return [dSdt, dIdt, dRdt]
12
13 # Parametros
14 N = 10000          # populacao total
15 beta = 0.5         # taxa de transmissao (1/dia)
16 gamma = 0.1        # taxa de recuperacao (1/dia)
17 R0 = beta/gamma    # R0 = 5
18
19 # Condicoes iniciais
20 I0 = 10
21 S0 = N - I0
22 R0_initial = 0
23 y0 = [S0, I0, R0_initial]
24
25 # Tempo
26 t = np.linspace(0, 200, 1000)
27
28 # Resolver
29 sol = odeint(sir_model, y0, t, args=(beta, gamma, N))
30 S, I, R = sol.T
```

Listing 5: Simulação de epidemia SIR

Análise da Epidemia

```
1 # Plotar
2 plt.figure(figsize=(12, 5))
3
4 plt.subplot(1, 2, 1)
5 plt.plot(t, S, 'b-', linewidth=2, label='Suscetíveis (S)')
6 plt.plot(t, I, 'r-', linewidth=2, label='Infectados (I)')
7 plt.plot(t, R, 'g-', linewidth=2, label='Recuperados (R)')
8 plt.xlabel('Tempo (dias)')
9 plt.ylabel('Numero de individuos')
10 plt.title(f'Dinamica SIR (R0 = {R0:.1f})')
11 plt.legend()
12 plt.grid(True, alpha=0.3)
13
14 # Encontrar pico da epidemia
15 peak_idx = np.argmax(I)
16 peak_time = t[peak_idx]
17 peak_infected = I[peak_idx]
18
19 print(f"Pico de infectados: {peak_infected:.0f} individuos")
20 print(f"Tempo do pico: {peak_time:.1f} dias")
21 print(f"Taxa de ataque final: {R[-1]/N*100:.1f}%")
22
23 # Limiar epidemico
24 S_threshold = N / R0
25 print(f"Limiar de suscetiveis: {S_threshold:.0f}")
26
27 plt.subplot(1, 2, 2)
28 plt.plot(S, I, 'purple', linewidth=2)
29 plt.axvline(x=S_threshold, color='red', linestyle='--',
30            label=f'S = N/R0 = {S_threshold:.0f}')
31 plt.xlabel('Suscetiveis (S)')
32 plt.ylabel('Infectados (I)')
33 plt.title('Retrato de Fase')
34 plt.legend()
35 plt.grid(True, alpha=0.3)
```

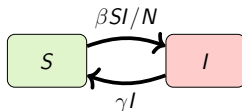
Listing 6: Análise de resultados

Modelo SIS

SIS: Sem Imunidade Permanente

Indivíduos recuperados voltam a ser susceptíveis (ex: resfriado comum, gonorreia):

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} + \gamma I \\ \frac{dI}{dt} &= \beta \frac{SI}{N} - \gamma I\end{aligned}$$



Equilíbrio endêmico: Se $R_0 > 1$,

$$I^* = N \left(1 - \frac{1}{R_0} \right), \quad S^* = \frac{N}{R_0}$$

Importante!

Doença persiste permanentemente na população (endemia)!

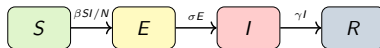
Modelo SEIR

SEIR: Com Período de Incubação

Adiciona compartimento de Expostos (infectados mas não infecciosos):

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} \\ \frac{dE}{dt} &= \beta \frac{SI}{N} - \sigma E \\ \frac{dI}{dt} &= \sigma E - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

onde σ = taxa de progressão ($1/\sigma$ = período de incubação).



R_0 modificado:

$$R_0 = \frac{\beta}{\gamma} \quad (\text{permanece o mesmo})$$

mas a dinâmica temporal é alterada (pico mais tardio).

Extensões do Modelo SIR

Demografia:

- Nascimentos: μN
- Mortes naturais: $\mu S, \mu I, \mu R$
- Mortalidade pela doença: αI

Estrutura etária:

- Diferentes β por faixa etária
- Matrizes de contato
- Vacinação por idade

Intervenções:

- Vacinação: νS
- Quarentena: redução de β
- Tratamento: aumento de γ
- Distanciamento social

Espaço:

- Modelos metapopulacionais
- Difusão espacial
- Redes de contato

Modelagem de COVID-19

Combinou SEIR + demografia + estrutura etária + intervenções não-farmacêuticas + vacinação

Metapopulações

Definição: Metapopulação

Conjunto de populações locais conectadas por dispersão, onde cada população local pode sofrer extinção e recolonização.

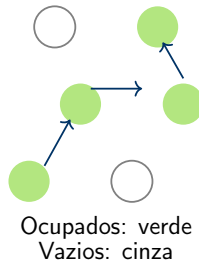
Modelo de Levins:

$$\frac{dp}{dt} = cp(1 - p) - ep$$

- p = fração de patches ocupados
- c = taxa de colonização
- e = taxa de extinção local

Equilíbrio:

$$p^* = 1 - \frac{e}{c}$$



Condição de persistência:

$$c > e$$

Equações de Reação-Difusão

Difusão Espacial

Combina crescimento local com dispersão espacial:

$$\frac{\partial N}{\partial t} = \underbrace{f(N)}_{\text{reação}} + \underbrace{D \nabla^2 N}_{\text{difusão}}$$

Em 1D:

$$\frac{\partial N}{\partial t} = f(N) + D \frac{\partial^2 N}{\partial x^2}$$

Parâmetros:

- D = coeficiente de difusão (taxa de dispersão)
- $f(N)$ = dinâmica local (ex: logística)
- ∇^2 = operador Laplaciano

Exemplo: Equação de Fisher-KPP

$$\frac{\partial N}{\partial t} = rN \left(1 - \frac{N}{K}\right) + D \frac{\partial^2 N}{\partial x^2}$$

Modelo de invasão espacial de uma espécie

Ondas Viajantes

Solução de Onda Viajante

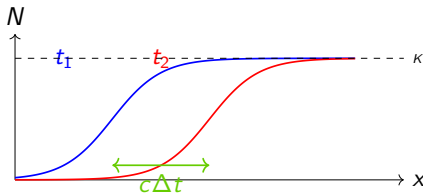
Perturbação que se propaga no espaço mantendo sua forma:

$$N(x, t) = N(z), \quad z = x - ct$$

onde c é a velocidade da onda.

Velocidade mínima (Fisher-KPP):

$$c_{min} = 2\sqrt{rD}$$



Aplicações: Invasões biológicas, frentes de epidemias, propagação de genes

Padrões de Turing

Instabilidade de Difusão

Sistema de duas espécies com difusão diferencial pode gerar padrões espaciais espontâneos:

$$\begin{aligned}\frac{\partial u}{\partial t} &= f(u, v) + D_u \nabla^2 u \\ \frac{\partial v}{\partial t} &= g(u, v) + D_v \nabla^2 v\end{aligned}$$

Condição para padrões: $D_v \gg D_u$ (inibidor difunde mais rápido que ativador)

Mecanismo:

- Ativador de curto alcance
- Inibidor de longo alcance
- Desestabilização por difusão

Exemplos biológicos:

- Padrões de pelagem animal
- Distribuição de estômatos em folhas
- Arranjo de penas em aves



Padrão de manchas

Importante!

Alan Turing (1952): "The Chemical Basis of Morphogenesis"

Implementação: Difusão 1D

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def fisher_kpp_1d(N, t, r, K, D, dx):
5     """Equacao de Fisher-KPP em 1D"""
6     # Termo de reacao (logistico)
7     reaction = r * N * (1 - N/K)
8
9     # Termo de difusao (diferenca finita central)
10    diffusion = np.zeros_like(N)
11    diffusion[1:-1] = D * (N[2:] - 2*N[1:-1] + N[:-2]) / dx**2
12
13    # Condicoes de contorno (sem fluxo)
14    diffusion[0] = diffusion[-1] = 0
15
16    return reaction + diffusion
17
18 # Parametros
19 L = 50          # tamanho do dominio
20 nx = 200        # pontos espaciais
21 x = np.linspace(0, L, nx)
22 dx = x[1] - x[0]
23
24 r = 0.5         # taxa de crescimento
25 K = 1.0         # capacidade de suporte
26 D = 0.1         # coeficiente de difusao
27
28 # Condicao inicial (pulso localizado)
29 N0 = np.zeros(nx)
30 N0[nx//4:nx//4+10] = K
31
32 # Simular usando metodo de Euler
33 t_max = 100
34 dt = 0.01
35 steps = int(t_max / dt)
36
37 N = N0.copy()
38 for step in range(steps):
39     N += dt * fisher_kpp_1d(N, step*dt, r, K, D, dx)
```

Listing 7: Equação de Fisher-KPP em 1D

Metapopulações e Redes

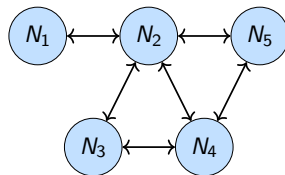
Modelo em Rede:

$$\frac{dN_i}{dt} = f(N_i) + \sum_{j=1}^n m_{ij}(N_j - N_i)$$

- N_i = população no patch i
- m_{ij} = taxa de migração de i para j
- $f(N_i)$ = dinâmica local

Sincronização:

- Alta conectividade $\rightarrow$ sincronismo
- Baixa conectividade $\rightarrow$ assincronia
- Importante para estabilidade



Rede de patches

Topologias:

- Regular (grade)
- Aleatória
- Escala-livre
- Mundo-pequeno

Aplicação

Conservação: corredores ecológicos conectando fragmentos de habitat

Exercícios - Parte 1

Questão 1: Crescimento Populacional

Uma população segue o modelo logístico com $r = 0.6 \text{ dia}^{-1}$ e $K = 5000$.

1. Se $N(0) = 500$, calcule $N(10)$
2. Em que tempo a população atinge 90% de K ?
3. Qual a taxa máxima de crescimento (dN/dt)? Em que densidade?

Questão 2: Efeito Allee

Considere o modelo com efeito Allee: $\frac{dN}{dt} = rN(1 - N/K)(N/A - 1)$, com $r = 0.5$, $K = 1000$, $A = 100$.

1. Encontre os pontos de equilíbrio
2. Analise a estabilidade de cada equilíbrio
3. Simule em Python para $N(0) = 50$ e $N(0) = 150$
4. Qual a implicação para conservação?

Exercícios - Parte 2

Questão 3: Modelo de Ricker

Implemente o modelo de Ricker: $N_{t+1} = N_t e^{r(1-N_t/K)}$

1. Simule para $r = 1.5, 2.0, 2.5, 3.0$ com $K = 100$
2. Crie diagramas de bifurcação variando r de 0 a 3
3. Identifique regiões de estabilidade e caos
4. Compare com o modelo de Beverton-Holt

Questão 4: Lotka-Volterra

Para o sistema predador-presa com $\alpha = 1.0$, $\beta = 0.1$, $\delta = 0.75$, $\gamma = 1.5$:

1. Calcule o ponto de equilíbrio não-trivial
2. Calcule a matriz Jacobiana no equilíbrio
3. Determine os autovalores. O equilíbrio é estável?
4. Simule e plote séries temporais e retrato de fase

Exercícios - Parte 3

Questão 5: Modelo SIR

Uma doença tem $\beta = 0.4 \text{ dia}^{-1}$ e $\gamma = 0.1 \text{ dia}^{-1}$ em uma população de $N = 10000$.

1. Calcule R_0
2. Se inicialmente $I(0) = 20$, simule 200 dias
3. Qual o pico de infectados? Quando ocorre?
4. Qual a taxa de ataque final (proporção que adoece)?
5. Que fração deve ser vacinada para prevenir epidemia?

Projeto: Competição e Coexistência

Implemente o modelo de competição Lotka-Volterra:

- Simule os 4 cenários (sp1 vence, sp2 vence, coexistência, bistabilidade)
- Crie retratos de fase com isoclinas
- Adicione estocasticidade demográfica
- Explore efeitos de flutuações ambientais

Referências Principais

-  Murray J.D. (2002). *Mathematical Biology I: An Introduction*, 3rd ed. Springer.
-  Edelstein-Keshet L. (2005). *Mathematical Models in Biology*. SIAM.
-  Otto S.P., Day T. (2007). *A Biologist's Guide to Mathematical Modeling in Ecology and Evolution*. Princeton University Press.
-  Keeling M.J., Rohani P. (2008). *Modeling Infectious Diseases in Humans and Animals*. Princeton University Press.
-  Anderson R.M., May R.M. (1991). *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press.

Leitura Complementar

Ecologia Teórica

- Case T.J. (2000). *An Illustrated Guide to Theoretical Ecology*
- Kot M. (2001). *Elements of Mathematical Ecology*
- Tilman D. (1982). *Resource Competition and Community Structure*

Epidemiologia

- Diekmann O., Heesterbeek H. (2000). *Mathematical Epidemiology of Infectious Diseases*
- Brauer F., Castillo-Chavez C. (2012). *Mathematical Models in Population Biology and Epidemiology*

Recursos Computacionais

- **EpiModel**: <https://epimodel.org> (R package para epidemiologia)
- **PyDSTool**: <https://pydstool.github.io> (Python para sistemas dinâmicos)
- **COPASI**: <http://copasi.org> (simulação de modelos)

Obrigado!

Capítulo 10: Sistemas Populacionais

Próximo Capítulo:
Tópicos Avançados em Biologia de Sistemas

Dúvidas? Perguntas?